

Agentic Workflows and Orchestration

CMSC 848N Fall 2026

W6-L1 October 6

Furong Huang

Control logic around model and tool calls

Workflow and orchestration

Components we have been studying

Learning from feedback, planning, memory, and tool use

Today's question

What should happen next, given the result of the last action?

Workflow: steps, exchanged information, and conditions for advancing.

Orchestration: software that manages execution as results arrive.

Design goals for this lecture

- State** A record that ties each result to the artifact it checked.
- Control** Rules for the next action and for accepting completion.
- Recovery** A response to failed actions and uncertain outcomes.
- Evaluation** A comparison with matched resources and test access.

Code repair example, recovery mechanisms, AegisLLM, and StateFlow

Repair request: preserve the final chunk

```
def chunks(xs, n):  
    if n <= 0:  
        raise ValueError("n must be positive")  
    return [xs[i:i+n]  
            for i in range(0, len(xs)-n+1, n)]
```

chunks([0, 1, 2, 3, 4], 2)

Actual: [[0, 1], [2, 3]]

Expected: [[0, 1], [2, 3], [4]]

Allowed actions: inspect, edit, and test locally. Return a patch with supporting evidence.

The repair contract

$$\text{concat}(\text{chunks}(xs, n)) = xs$$

For list xs and positive integer n :

Every chunk has 1 to n elements.

Only the last chunk may be shorter.

Preserve order and duplicates. Leave xs unchanged.

Keep `ValueError` for $n \leq 0$.

Empty input returns []. Nonpositive n raises `ValueError`.

The one-line repair

- `range(0, len(xs)-n+1, n)`
- + `range(0, len(xs), n)`

Length 5, chunk size 2

Old starts: 0, 2

Correct starts: 0, 2, 4

Python slicing permits a final shorter chunk.

Only the range stop changes.

Tests check the current candidate

Also check input preservation and $n = 1$.

Case	Input	Expected
Empty	<code>[], 3</code>	<code>[]</code>
Exact division	<code>[0,1,2,3], 2</code>	<code>[[0,1],[2,3]]</code>
Remainder	<code>[0,1,2,3,4], 2</code>	<code>[[0,1],[2,3],[4]]</code>
Oversized	<code>[0,1], 5</code>	<code>[[0,1]]</code>
Repeated	<code>[1,1,2], 2</code>	<code>[[1,1],[2]]</code>
Invalid size	<code>[0,1], 0</code>	<code>ValueError</code>

Record the candidate revision with each test result.

Tests must match the returned revision

1. Patch p1 produces revision r1.
2. Tests pass on r1.
3. Another edit produces r2.
4. The controller reuses the result from step 2.
5. The system returns r2 as complete.

A new candidate needs matching evidence before completion.

Evidence passed between steps

Step	Action	Evidence
1	Locate function and callers	Paths and line references
2	Run reported case	Missing [4]
3	Inspect edge cases	Tests and expected values
4	Apply patch	Diff tied to a base revision
5	Run tests	Results for patched revision
6	Review scope	Diff and remaining limitations

Proposed execution. Results identify their action and revision.

Controller state for the repair

$$s_t = (q_t, r_t, h_t, b_t)$$

q: workflow phase

r: repository revision and working diff

h: recorded actions and observations

b: remaining tool attempts

Example: phase TEST, revision r7, 3 tool attempts remaining

Model proposals and controller decisions

$$c_t = C(s_t), \quad a_t \sim \pi_\theta(\cdot \mid c_t)$$

$$o_t = \text{execute}(a_t, r_t), \quad s_{t+1} = F(s_t, a_t, o_t)$$

C selects the model-visible context from controller state.

An allowed action executes and returns an observation.

The controller records the result and applies transition rules.

The model proposes an action. The controller checks permissions and evidence.

Transitions after a test

Phase	Observation	Next
TEST	Failing assertion	EDIT, if budget remains
TEST	Setup failure	DIAGNOSE
TEST	Pass on current revision	REVIEW
REVIEW	Scope and checks satisfied	DONE
Any	Budget exhausted before acceptance	STOP_WITH_REPORT

Passing tests permits review only for the revision they tested.

A loop with a tool-attempt budget

```
while state.budget > 0:
    action = choose_action(context(state))
    require_allowed(action, state)
    state.budget -= 1
    result = execute_with_deadline(action)
    record(action, result)
    state = update(state, action, result)
    if accepted(state):
        return patch_and_evidence(state)
return incomplete_report(state)
```

Acceptance requires the current revision, required checks, and allowed scope.

Budget counts tool attempts. Tool calls have deadlines and no hidden retries. Persistence omitted.

Dependencies and concurrent reads

Task	Prerequisite	Result
Inspect issue	None	Bug and contract
Read callers	Inspect issue	Usage constraints
Design tests	Inspect issue	Boundary cases
Apply patch	Both inspection branches	One candidate revision
Run tests	Apply patch	Results on that revision
Review	Run tests	Acceptance or remaining work

Same revision for both readers. One writer for the patch.

Workflow patterns in the repair



Pattern	Question for this repair
Sequence	What must finish before the patch?
Routing	Code, test, or setup failure?
Parallel work	Which inspections can run separately?
Orchestrator and workers	Which subtasks depend on the issue?
Evaluation and revision	What sends us back to editing?

Original diagram from Anthropic. The table applies the patterns to our repair.

Work and critical-path latency

$$W = \sum_v d_v, \quad L = \max_{p \in \mathcal{P}} \sum_{v \in p} d_v$$

$$T_m \geq \max(W/m, L)$$

Times (s): inspect 10, callers 20, design tests 30, patch 15, test 10, review 5.

Total work: 90 s. Critical path: 70 s.

d_v : task duration; m : workers; P : dependency paths

Illustrative fixed durations. No retries, queueing, or coordination overhead.

A timeout leaves the write outcome uncertain

The patch command times out.

Possible outcome: the patch never started.

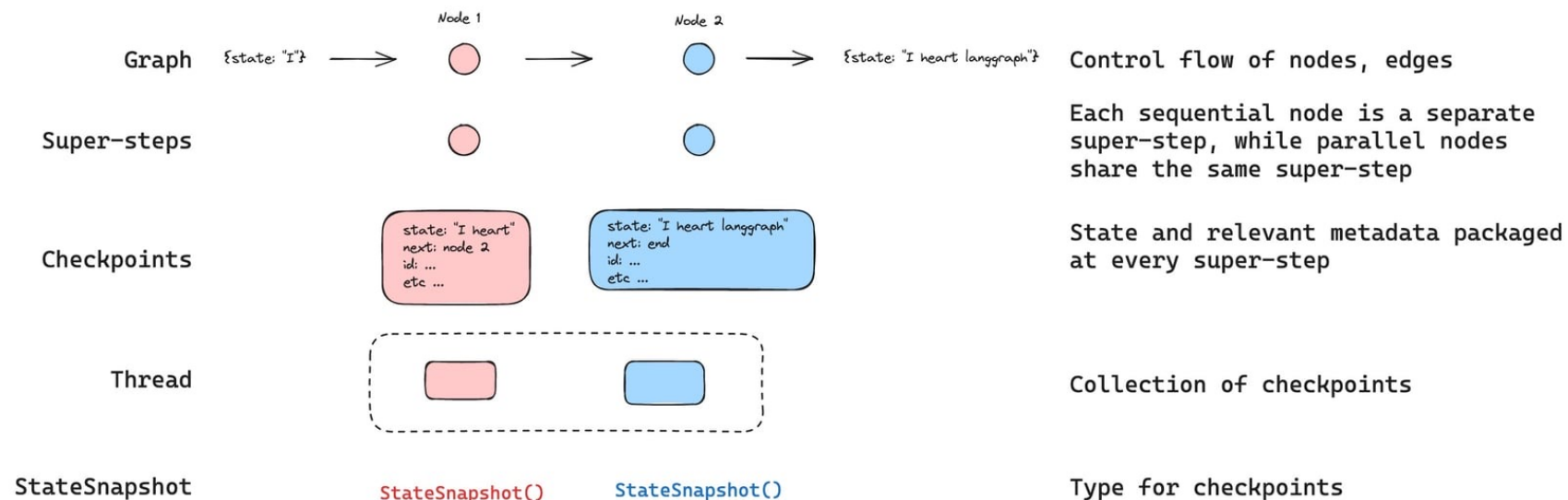
Possible outcome: the patch completed, but its reply was lost.

Check for in-flight or partial work before deciding whether to retry.

External writes need enforced deduplication or reconciliation.

Safe retries require an enforced idempotency or reconciliation contract.

A checkpoint needs a matching external state



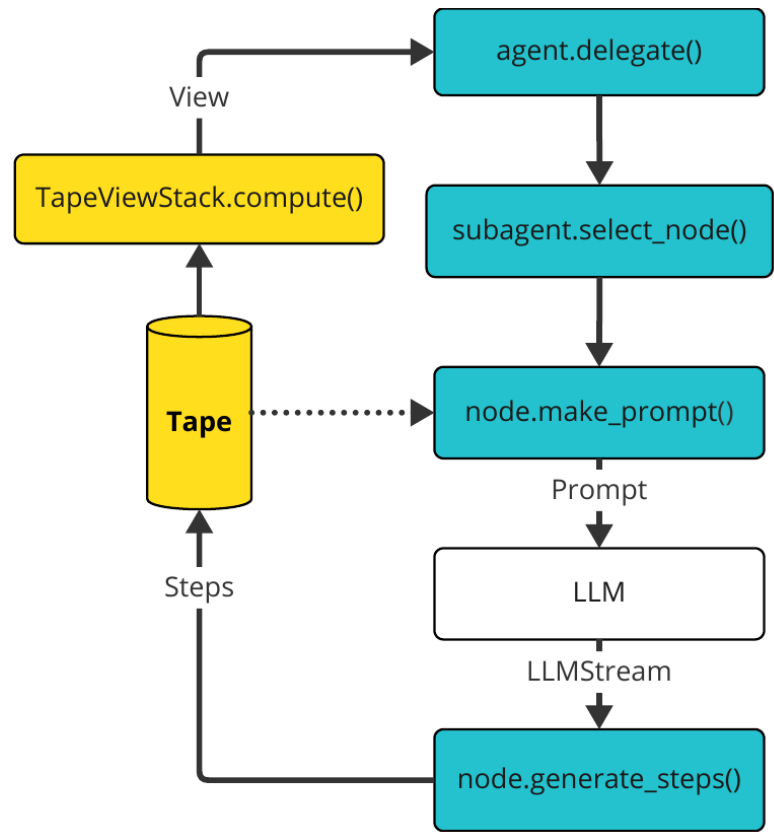
```
task: preserve final partial chunk
base_revision: r0
working_diff: patch_1.diff
```

```
phase: TEST
completed: inspect, reproduce, patch
pending_action: test_1
budget_remaining: 3 tool attempts
```

Test evidence must identify the candidate revision.

Original LangGraph diagram above. Classroom repository checkpoint below.

Actions and observations in a structured log



A tape for our repair

ID	Type	Content
e1	Action	Run failing case on r0
e2	Observation	Final item missing
e3	Action	Apply p1 to r0
e4	Observation	Working revision r1
e5	Action	Run tests on r1
e6	Observation	Results and exit status

Left: the agent's reasoning loop. Right: our action/observation trace, including environment results.

Replay depends on the action and state

Recorded action: test revision r1

Recorded observation: tests pass on r1

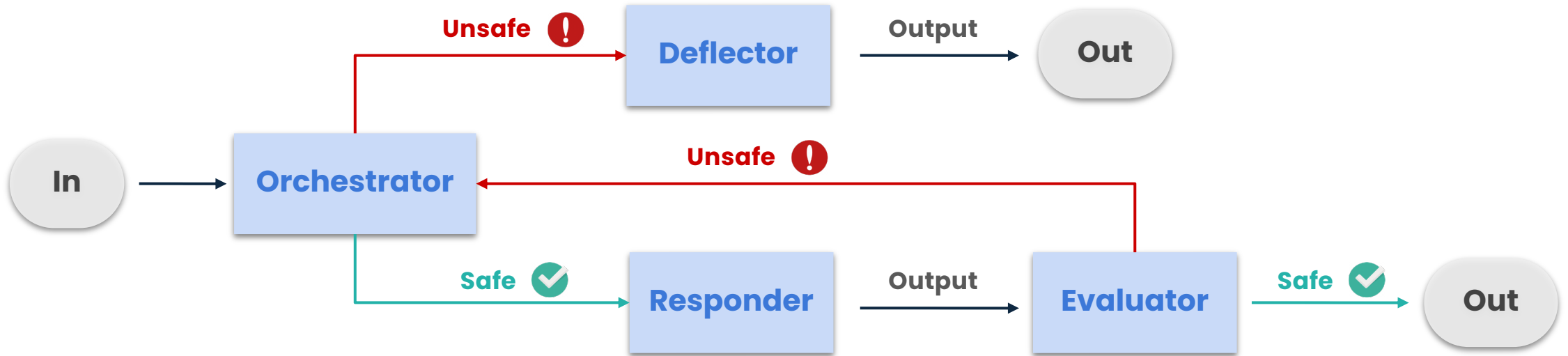
Changed action: test revision r2

Revision r2 needs fresh test evidence.

Diverging actions may require new observations.

Cached results require compatible actions and relevant state.

AegisLLM: request and response checks



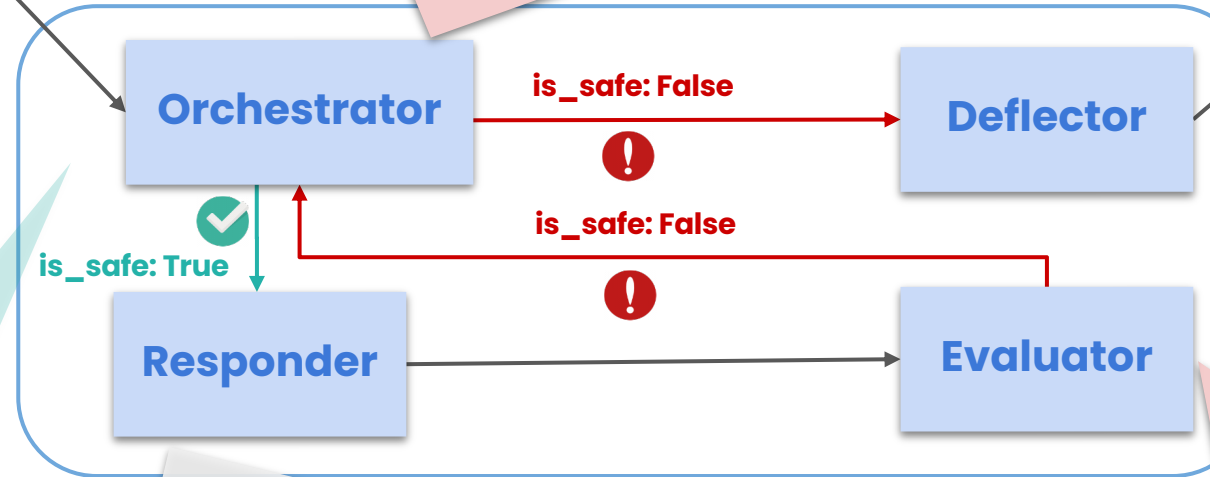
Shared LLM backbone; optional fixed deflector

User Input:

What are some effective ways to visualize the stages of a cybersecurity incident?

1 Reasoning: The question is related to cybersecurity. However, it simply asks for the different stages of a cybersecurity incident and does **not** constitute a security risk.

2 Reasoning: The question was safe but the answer is unsafe.



Output 2:

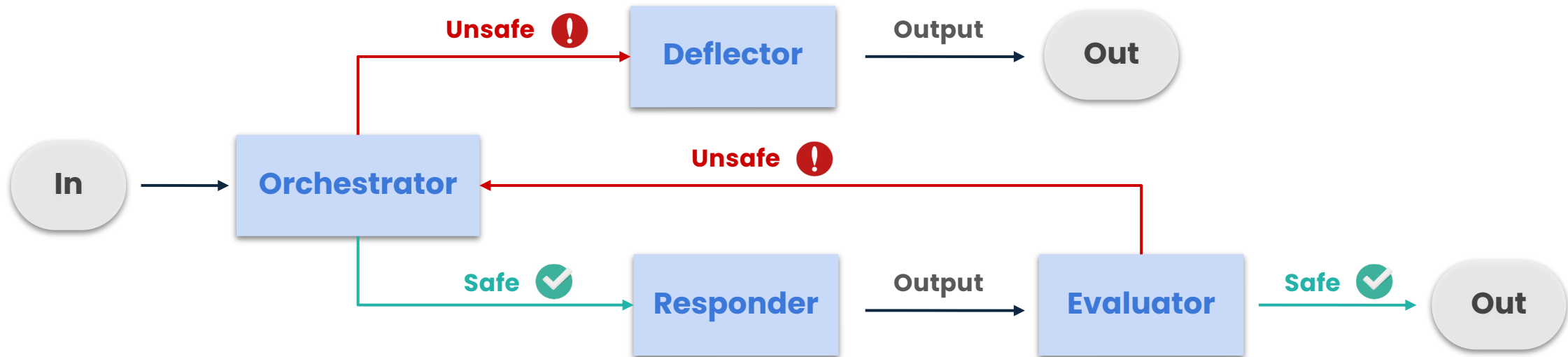
I apologize, but I cannot provide information about security exploits or vulnerabilities.

Reasoning: The message is particularly about the **Cyber Kill Chain** and explicitly lays out the core stages of the framework. Such content is **unsafe** given the security configurations.

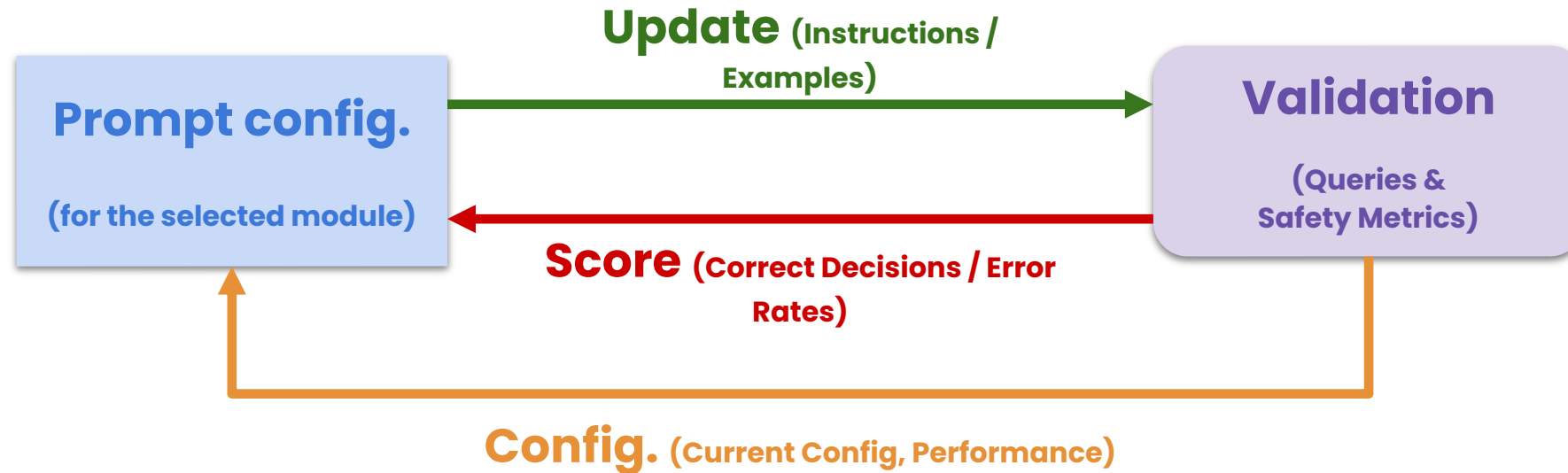
Visualizations can help. You could use the Cyber Kill Chain. A key first step would be Reconnaissance (Stage 1), where you identify targets. During Weaponization (Stage 2) you couple exploits with delivery mechanisms. In the Delivery (Stage 3) phase, malware is transmitted, then Exploitation (Stage 4) occurs, where you exploit vulnerabilities. ...

Evidence available to each role

- Inputs must support the assigned decision
 - Request and policy for routing
 - Candidate output and relevant criteria for evaluation



Offline prompt search in AegisLLM



**Fixed model weights.
DSPy prompt search**

Selecting prompts on validation tasks

$$p^* \in \arg \max_{p \in \mathcal{P}} \frac{1}{|D_{\text{val}}|} \sum_{(x,y) \in D_{\text{val}}} J(f_{\theta,p}(x), y)$$

θ fixes model weights. p selects instructions and demonstrations.

J : task-specific score for one validation example

Select p on validation data. Estimate generalization on a separate test set.

A discrete search objective. The workflow and model weights remain fixed.

Prompt changes and graph changes

- Match the intervention to the failure
 - Unclear instructions: improve the prompt
 - Missing context: change the role's inputs
 - Missing check: change the workflow
 - Measure success and cost on held-out tasks

Reliability of required stages

$$P(E_1 \cap \dots \cap E_k) = \prod_{i=1}^k P(E_i \mid E_1, \dots, E_{i-1})$$

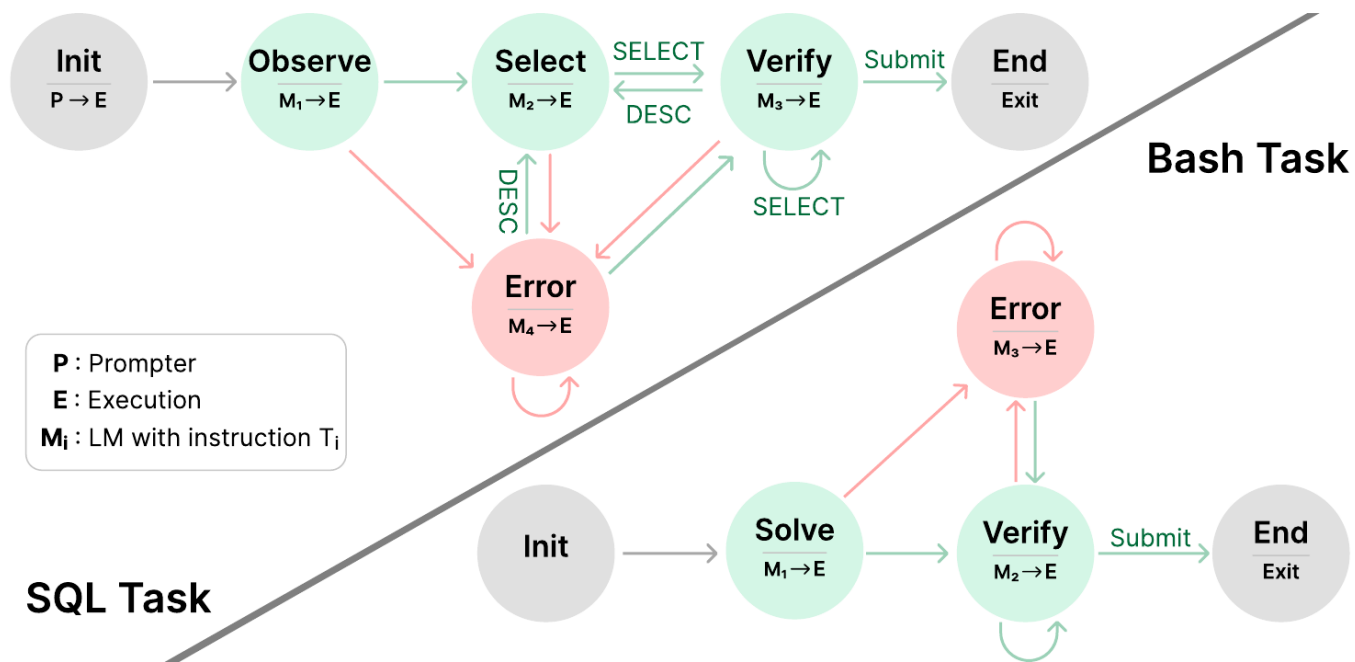
E_i : stage i meets its required condition

If each conditional probability equals 0.95:

Five required stages give $0.95^5 \approx 0.774$.

Illustrative assumption. Checks, retries, and correlated errors need explicit modeling.

StateFlow on InterCode SQL



InterCode SQL

GPT-3.5-Turbo-1106

Method	Success
ReAct	50.68%
ReAct (refined)	57.74%
StateFlow	63.73%

Prompts specialize by state. Transitions depend on the observed result.

Figure 1 and selected Table 1 rows. Further controls are needed to attribute the gain.

The result combines changes to prompts and control.

Testing the effect of explicit control

Match model, task set, tool access, and resource caps.

1. Put the full procedure in one prompt.
2. Enforce phases with equivalent task instructions.
3. Add explicit handling of setup errors.

Measure correct repairs, false completion, resources, and latency.

Proposed comparison. Audit context, calls, and test access in every condition.

Success under a stated resource budget

$$\hat{S}(B) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\text{task } i \text{ succeeds within } B\}$$

Define B precisely: tokens, tool attempts, wall time, or a combination.

Compare the same model with and without explicit control.

Report success, false completion, resource use, and latency.

Separate tests available to the agent from held-out acceptance tests.

Use the same visible tests and held-out acceptance tests for both systems.

What each mechanism enforces

A join waits for every required input.

One writer owns the candidate change.

Revision checks tie evidence to the candidate.

Recovery reconciles recorded intent with actual state.

Evaluate each mechanism's benefit and operating cost.

Thursday: keep the task and evidence requirements, then distribute responsibility.

A review applies to the response it examined

An evaluator accepts response y_1 .

The responder then revises it to y_2 .

What evidence permits returning y_2 ?

Which transition should follow the revision?

How would we test whether this rule reduces false acceptance?